HID 위장 장치의 키 입력 복원 및 실행 의미 분석을 통한 BadUSB 탐지 기법

한우혁*, 곽나영*, 정지윤*, 정유찬*, 현상원†

,† 명지대학교 컴퓨터정보통신공학부 컴퓨터공학전공(학부생, 교수†)

BadUSB Attack Detection Based on Keystroke Reconstruction and Command Semantics Analysis

Woohyeok Han*, Nayeong Kwak*, Jiyoon Jung*, Yuchan Jung*, Sangwon Hyun†

*,† Dept. of Computer Engineering, Myongji University
(Undergraduate Student*, Professor†)

요 약

USB 키보드는 연결할 때 별도의 인증을 거치지 않고 입력의 신뢰성을 인정받는다. 이러한 구조를 악용하여 HID로 위장한 장치가 키 입력을 주입하는 BadUSB 공격은 공개된 이후 상당한 기간이 지난 현재까지도 여전히 보고되고 있다. 기존 방어 기법은 장치 단위의 허용과 차단, 또는 명령어 문자열 매칭에 머물러 있어 정상 키보드로 인식된 장치나 인코딩된 명령에는 대응하기 어렵다. 본 논문에서는 미승인 HID 장치로부터 주입된 키 입력을 실제 입력 문자열로 복원하고, 인코딩과 중첩 구조를 해석하여 얻은 명령어와 인자의 실행 의미를 분석하고, 단축키, URL 및 키 입력 간격의 위험도를 복합적으로 반영하여 위험도를 산출하는 기법을 제안한다. 제안 기법은 악성 여부에 대한 이진 판정이 아니라 위험도 점수와 판단 근거를 제시하며, 탐지 경보는 이 점수에 임계값을 적용하여 생성한다.

I. 서론

BadUSB는 저장장치나 케이블처럼 보이는 장치의 펌웨어를 조작하여, 원래와 다른 종류의 장치로 위장하여 동작하게 만드는 공격 기법이다[1]. USB 규격에서 장치의 종류는 장치가 스스로 보고한 디스크립터로 결정되며 호스트 운영체제는 이를 검증 없이 받아들인다. 따라서 펌웨어를 조작하면 장치를 키보드나 네트워크 어댑터 같은 다른 종류의 장치로 위장할 수 있다. 이 가운데 키보드와 같은 HID(Human Interface Device)로 위장하는 방식은 임의의 키 입력을 주입할 수 있으며, 주입된 입력이 호스트에서 사용자가 직접 누른 키와 구분되지 않기 때문에 위협이 크다. BadUSB 공격이 공개된 이후 상당한 기간이 지난 현재까지도, 연결된 웹캠의 펌웨어를 원격으로 변조하여 키보드로 동작시킬 수 있는 취약점이 보고되는 등 동일한 구조적 취약성을 이용한 공격이 계속 확인되고 있다[2].

이러한 공격에 대한 대응 방안은 크게 두 갈래로 제안되었다. 하나는 USB 장치를 신뢰할지 여부를 판단하는 접근으로, 장치의 기능을 사용자 기대와 비교하거나 연결 시점에 사용자의 명시적 승인을 요구하는 방식 등이 있다. 그러나 이들은 장치가 연결되는 시점을 대상으로 하므로 승인 이후에 주입되는 입력의 내용은 검사하지 않으며, 공격 장치가 정상 키보드와 같은 디스크립터를 보고하면 디스크립터 정보만으로는 두 장치를 구분할 수 없다. 다른 하나는 장치로부터 주입된 입력의 내용을 검사하는 접근이다. 기존 연구는 장치로부터 주입된 내용 중에 PowerShell 문자열이 포함되어 있는지와 키 입력 간격을 기준으로 HID 위장 공격을 탐지하는 알고리즘을 제안하였다[3].

그러나 문자열 매칭을 통해 특정 문자열의 포함 여부를 확인하는 방식은 명령어의 표현 형태에 의존한다는 한계가 있다. 공격 명령은 Base64 인코딩이나 문자열 분할과 결합, 중첩 호출을 통해 동작을 유지한 채 표현만 바꿀 수 있으며, 운영체제에 기본 포함된 실행 파일, 즉 LOLBin(Living-off-the-Land Binary)을 이용하면 셸 이름을 전혀 드러내지 않고도 동일한 공격을 구성할 수 있다[5].

본 논문에서는 USB 장치로부터 주입된 명령어에

† 교신저자: 현상원 (명지대학교 컴퓨터공학전공 교수)

이 성과는 정부(과학기술정보통신부)의 재원으로 한국연구재단의 지원을 받아 수행된 연구임 (RS-2026-25480989).

대한 단순 문자열 매칭이 아닌 명령어의 실행 의미를 분석하여 위험도를 평가하는 기법을 제안한다. 장치로부터 주입된 키 입력을 실제 입력 내용으로 복원한 뒤 인코딩과 중첩 구조를 해석하여 명령의 실행 의미를 파악하고 그에 따라 위험도를 점수화한다. 이를 통해 표현이 달라지거나 셸 이름이 드러나지 않는 명령에도 대응할 수 있다. 또한 제안하는 시스템의 분석 결과는 악성 여부에 대한 이진 판정이 아니라 위험도 점수와 판단 근거로 산출되며, 탐지 경보는 이 위험도 점수에 대한 임계값을 적용하여 생성한다.

II. 제안 기법

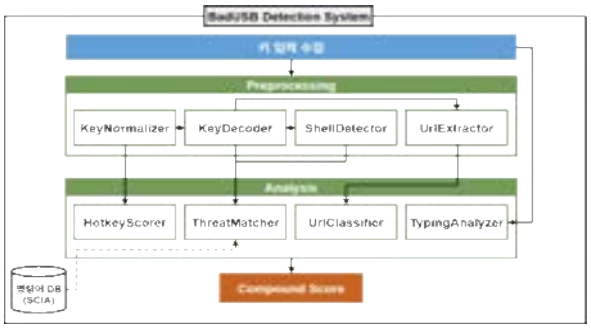


그림 1. 시스템 아키텍처

분석 대상은 운영체제에 HID 키보드로 등록된 장치가 주입한 키 입력이다. 대상 운영체제는 Windows이며 셸 환경은 CMD와 PowerShell로 한정한다. 키 배열은 US 키보드를 기준으로 하고, 세션은 미승인 HID 장치가 후킹 구간 동안 주입한 입력 버퍼 전체로 정의한다. 전체 구조는 [그림 1]과 같이 주입된 입력을 복원하는 Preprocessing과 복원된 내용을 평가하는 Analysis 두 단계로 구성된다.

2.1 Preprocessing

KeyNormalizer는 미승인 HID 장치가 연결된 구간에서 수집한 가상 키 코드(Virtual-Key Code) 열을 입력받아 실제 입력 문자열과 HotkeyEvent로 복원한다. 키다운, 키업 정보를 이용해 Shift, Ctrl, Alt, Win 등 조합 키(modifier)의 눌림 상태를 추적한다. 키업 이벤트는 조합 키의 해제 상태를 갱신하는 데에만 사용하며 문자로 변환하지 않는다. 키다운 이벤트는 Ctrl, Alt, Win 중 하나 이상이 눌려 있으면 HotkeyEvent로, 그렇지 않으면 일반 문자열로 분기한다. Shift는 문자 변환을 위한 조합 키이므로 분기 기준에서 제외한다.

조합 키가 눌린 상태의 입력은 일반 문자열에서 배제하고 HotkeyEvent로 분리한다. Win+X 와 같이 하위 메뉴를 여는 단축키는 뒤따르는 메뉴 선택 키까지 하나의 이벤트로 묶는다. 각각 별개의 이벤트로 기록하면 어떤 도구가 실행되었는지 확인할 수 없기 때문이다. Ctrl+Shift+Enter는 입력한 명령을 관리자 권한으로 실행하여 새로운 셸을 여는 단축키이므로, HotkeyEvent로 기록하는 동시에 이후 입력을 앞선 입력과 다른 줄로 분리한다.

조합 키가 눌리지 않은 입력에는 US 키보드의 Shift 치환 테이블을 적용하여 문자로 변환한다. Enter는 줄 단위 분리 기준으로 사용하고, Backspace는 현재 줄의 마지막 문자를 삭제하도록 반영하여 최종 입력을 복원한다. 복원된 줄 단위 문자열 목록과 HotkeyEvent 목록은 KeyDecoder로 전달된다.

KeyDecoder는 정규화된 각 줄을 체인 연산자 기준으로 분할해 명령어와 인자를 분리하며, 인자 내부에 중첩된 괄호 표현은 재귀적 파싱을 통해 은닉된 명령을 함께 추출한다. 이때 명령어는 소문자로 정규화하고, 인자는 원본 형태를 유지한다. Base64와 같이 대소문자 구분이 필요한 데이터가 손실되는 것을 막기 위함이다. 분리된 명령어와 인자는 ShellDetector와 ThreatMatcher로 전달된다.

ShellDetector는 KeyDecoder가 추출한 명령어와 단축키를 입력받아 이후 입력이 실행되는 셸 환경을 판정한다. 먼저 Win+X 하위 메뉴와 같이 셸을 직접 여는 단축키를 확인한다. Win+R로 열린 실행 창에 입력된 첫 명령어가 CMD 또는 PowerShell인지 확인한다. 단축키가 없는 경우에는 전체 명령어 중 CMD 또는 PowerShell이 처음 등장하는 지점을 셸 진입점으로 본다. 판정 결과는 CMD, PowerShell, 미감지 중 하나이며, ThreatMatcher가 SCIA 위험 정보 DB에서 CMD와 PowerShell 중 어느 셸의 항목과 매칭할지 결정하는 데 사용된다.

UrlExtractor는 KeyDecoder가 복원한 각 줄에서 URL을 추출한다. 먼저 http 또는 https로 시작하는 평문 URL을 탐색하고, Chr()이나 [char]와 같이 문자 코드로 표현된 부분을 실제 문자로 치환한 뒤 다시 탐색한다. URL이 여러 변수에 나누어 저장되거나 문자열 결합으로 생성되는 경우에는 변수 할당과 연결 연산을 순서대로 추적하여 최종 문자열로 복원한다. 추출된 URL은 퍼센트 인코딩을 디코딩한 형태로 저장되어 UrlClassifier로 전달된다.

2.2 Analysis

TypingAnalyzer는 키 입력 간격을 분석하여 사람의 타이핑과 자동화된 HID 입력을 구분하고, 그 결과를 바탕으로 타이핑 위험 점수를 산출한다. 조합 키를 제외한 키 입력 사이의 간격에 대해 중앙값과 MAD(Median Absolute Deviation)를 계산하며, 두 통계는 다운로드 대기와 같은 일시적인 긴 지연에 영향을 적게 받으므로 별도의 이상치 제거 없이 전체 구간에 적용할 수 있다. 입력 간격의 중앙값이 짧고 동시에 MAD가 작아 간격이 균일한 경우에만 자동화 입력으로 분류한다. 자동화로 분류되면 기본 점수를 부여한 뒤 간격이 짧고

균일할수록 점수를 가산하여 최대 10점까지 산출하고, 사람의 타이핑으로 분류되거나 입력 수가 부족해 판정할 수 없는 경우에는 0점을 부여한다.

ThreatMatcher는 명령어와 인자에 대해 SCIA 위험 정보 DB(이하 DB)를 기반으로 명령 위험 점수를 산출한다. SCIA는 CVSS 3.1의 S/C/I/A 구조를 차용하며, CIA는 기밀성(Confidentiality), 무결성(Integrity), 가용성(Availability)의 의미를 따르고, S(Scope)는 권한 상승 여부를 나타내도록 재정의하였다[4]. DB는 PowerShell과 CMD 명령을 명령어와 인자로 나누고, 인자는 다시 옵션과 옵션 값으로 구분하여 SCIA 점수와 위협 태그를 부여한다.

SCIA 점수는 다음과 같이 정의된다. 무결성 위협(I)과 권한 상승(S)은 시스템 상태 또는 실행 권한에 직접적인 영향을 줄 수 있어 5.0점, 가용성 위협(A)은 서비스 사용에 피해를 줄 수 있어 4.0점, 기밀성 위협(C)은 단독으로 시스템의 상태나 권한을 직접 변경하지 않는다는 점을 고려하여 3.0점으로 설정하였다.

위협 태그는 명령을 구성하는 명령어와 옵션 및 옵션 값이 어떤 위협 행위에 해당하는지 나타내며, 앞서 정의한 SCIA 축 중 하나에 대응된다. 예를 들어 인코딩된 명령을 전달하는 '-EncodedCommand' 옵션은 Script-Payload 태그(I)에, 파일을 전송하는 '-OutFile' 옵션은 File-Transfer 태그(C)에 속한다. 각 위협 태그의 점수는 태그가 속한 SCIA 점수를 상한으로 하여 상대적 위험도에 따라 차등 부여된다. 이는 추후 나오는 session_score를 계산할 때 사용된다.

또한 특정 옵션과 옵션 값의 조합에 따라 위험도가 달라지는 경우 추가 점수를 반영한다. 예를 들어 PowerShell 명령어에 바로 붙는 '-ExecutionPolicy'(이하 ep) 옵션에는 Restricted(스크립트 실행 전면 차단), Unrestricted(경고 발생 및 실행 허용), Bypass(차단, 경고 없이 실행 허용) 등이 옵션 값으로 사용된다. 이 중 '-ep Bypass', '-ep Unrestricted' 조합은 모두 스크립트 실행을 허용하지만 경고 여부에 따라 각각 3.0점, 2.5점의 추가 점수를 부여한다. 이러한 옵션-옵션 값 조합은 DB에 별도로 저장된다.

$$session_score = \min(10,\ \text{session_raw} / \sqrt{N})$$

명령 위험 점수(session_score)는 DB 매칭으로 얻은 명령어, 옵션, 옵션 값별 점수를 합산한 값(session_raw)을, 매칭된 서로 다른 위협 태그 종류 수 N의 제곱근으로 나누어 산출된다. 이는 여러 종류의 저위험 태그가 중복 누적되어 위험도가 과대평가되는 것을 완화하고, 소수의 고위험 명령이 포함된 세션이 상대적으로 높은 점수를 받도록 한다.

UrlClassifier는 UrlExtractor에서 추출한 URL의 특성을 분석하여 URL 위험 점수를 산출한다. IP 주소 사용 여부, URL 단축 서비스, 실행 파일 확장자, 고엔트로피 도메인(DGA 의심), 비표준 포트, 동적 DNS, 무료 호스팅 도메인, 파일 공유와 터널링 서비스 등을 검사하여 해당하는 위험 특징을 표시한다. 각 특징에는 서로 다른 점수를 부여하고, 한 URL에서 확인된 특징들의 점수를 합산하여 해당 URL의 위험 점수로 사용한다. 이에 따라 정상으로 사용되는 경우가 많은 특징은 단독으로 위험 점수를 크게 높이지 않고, 다른 특징과 함께 나타날 때 위험 점수가 높아진다.

HotkeyScorer는 명령어로 나타나지 않아 ThreatMatcher가 포착할 수 없는 단축키 입력을 분석하여 단축키 위험 점수를 산출한다. Preprocessing에서 추출된 HotkeyEvent 중 실행 창 호출, 관리자 권한 실행, 시스템 관리 도구 실행과 같이 공격 진입에 사용되는 조합에 점수를 부여하고, Ctrl+C와 같이 정상 작업에서 흔히 쓰이는 조합에는 점수를 부여하지 않는다. Win+X는 하위 메뉴에서 관리자 권한 셸이 선택된 경우 추가 위험 점수를 부여한다. 여러 위험 단축키가 사용된 경우 점수를 누적하되 최대 10점으로 제한한다.

CompoundScorer는 앞서 설명한 TypingAnalyzer, ThreatMatcher, UrlClassifier, HotkeyScorer 4가지 분석 모듈에서 추출한 점수들을 종합하여 복합 위험도를 산출한다.

$$final_score = \min(10,\ 1.0 \cdot threat + 0.2 \cdot url + 0.3 \cdot typing + 0.5 \cdot hotkey)$$

final_score 식의 threat, url, typing, hotkey는 각각 ThreatMatcher, UrlClassifier, TypingAnalyzer, HotkeyScorer를 의미한다. 가중치는 각 분석 요소의 공격 판단 기여도와 우회 가능성을 고려하여 실험적으로 설정하였다. 명령의 실행 의미(threat)를 주요 판단 근거로 두어 가장 높은 가중치를 부여하고, 공격 진입 단축키(hotkey)를 그다음으로 설정하였다. 타이핑 및 URL 정보는 상대적으로 우회가 용이하므로 보조적인 판단 근거로 활용하였다.

제안 시스템은 탐지된 위협을 자동으로 차단하지 않고, 위험도를 등급화하여 사용자에게 차단 여부를 권고한다. CompoundScorer가 산출한 복합 위험도는 안전(3.0 미만), 보통(3.0 이상 5.0 미만), 경고(5.0 이상 7.5 미만), 위험(7.5 이상)의 4단계로 구분하며, 경고 이상을 차단 권고 대상으로 설정하였다.

III. 성능 평가

제안 시스템은 C++ 기반 Windows 엔드포인트 애플리케이션으로 구현하여 성능을 평가하였다. 평가 데이터셋은 키 입력 내용만 포함하고 실제 입력 타이밍 정보는 제공하지 않으므로, 배치 실행 시 입력 간격을 50ms로 고정하였다. 이에

본 평가는 입력 내용에 기반한 분석 요소인 명령어, 단축키, URL의 탐지 성능을 중심으로 수행하였으며, 실제 키별 타임스탬프가 필요한 TypingAnalyzer는 정량 평가에서 제외하였다.

사용되는 데이터셋은 크게 내부와 외부 두 종류로 나뉜다. 내부 데이터셋은 자체 제작한 BadUSB 페이로드 134건과 정상 HID 동작 99건, 총 233건으로 구성된다. 외부 데이터셋은 튜닝에 사용하지 않은 공개 저장소[1]의 Windows 대상 페이로드 196건이다. 이 중 악성 페이로드 121건이 평가에 활용되었다.

차단 권고 임계값을 1.0~8.0까지 조정하며 측정한 결과, 3.2~6.2 구간에서 F1-score가 최고치인 0.99로 유지되었다. 또한 5-fold 교차 검증에서 선정된 임계값의 평균이 4.9로 나타나 최종 임계값을 5.0으로 설정하였다.

3.1 탐지 성능 비교

구분	기법	Precision	Recall	F1
baseline	문자열 기반 탐지 (PowerShell)	1.00	0.54	0.71
baseline 확장	문자열 기반 탐지 (PowerShell+CMD)	1.00	0.96	0.98
제안 기법		0.99	0.99	0.99

표 1. 기존 기법과의 탐지 성능 비교 (233건)

[표 1]은 PowerShell 문자열 기반 탐지, CMD 키워드를 추가한 확장 문자열 기반 탐지, 제안 기법의 성능을 비교한 결과이다[3]. 제안 기법은 Precision 0.99, Recall 0.99, F1-score 0.99를 기록하였다. 추가적으로 외부 데이터셋으로 평가한 결과 악성 페이로드 121건 중 111건(91.7%)을 탐지하였다.

확장 문자열 기반 탐지가 높은 재현율을 보인 것은, 내부 데이터셋의 악성 페이로드가 PowerShell, CMD 키워드를 직접적으로 노출하기 때문이다. 그러나 LOLBin 기법을 이용하면, 셸 키워드 없이도 공격 명령을 구성할 수 있어 문자열 기반 탐지를 우회할 수 있다.

3.2 LOLBin 명령 탐지 성능 비교

이를 검증하기 위해 MITRE Atomic Red Team[2]에서 키워드 없는 LOLBin 명령 120건을 추출해 평가하였고, 결과는 [표 2]와 같다.

구분	기법	Recall
baseline	문자열 기반 탐지(PowerShell)[3]	0.00
baseline 확장	문자열 기반 탐지 (PowerShell+CMD)	0.02
제안 기법		0.93

표 2. 키워드 없는 LOLBin 명령에 대한 탐지율 (120건)

문자열 기반 탐지는 재현율 0.00 - 0.02로 대부분 탐지하지 못한 반면 제안 기법은 0.93을 유지하였다. 문자열 기반 탐지가 셸 키워드에 의존해 우회에 취약한 것과 달리, 제안 기법은 명령어, 인자를 SCIA 축으로 점수화하여 키워드가 없는 LOLBin 명령도 실행 의미를 근거로 탐지할 수 있다.

IV. 결론

본 논문에서는 미승인 HID 장치가 주입한 키 입력을 복원하고, 명령의 실행 의미를 SCIA 축으로 점수화하여 복합 위험도를 산출하는 BadUSB 탐지 기법을 제안하였다. 제안 기법은 문자열 기반 탐지가 아닌 명령의 실행 의미를 판단 근거로 삼아, 인코딩, LOLBin 등으로 키워드를 우회하는 공격까지 탐지한다는 점에서 기존 문자열 기반 탐지와 차별된다.

향후 연구에서는 실제 입력 타이밍을 수집하여 TypingAnalyzer의 성능을 검증하고, 수작업으로 구축한 명령어 DB를 AI로 자동으로 분류하고 갱신하여 새로운 명령과 변형 기법까지 탐지 범위를 넓히고자 한다.

[참고문헌]

[1] K. Nohl and J. Lell, "BadUSB: On Accessories that Turn Evil," Black Hat USA, Las Vegas, NV, Aug. 2014.

[2] J. Michael and M. Shkatov, "BadCam: Now Weaponizing Linux Webcams," DEF CON 33, Las Vegas, NV, Aug. 2025.

[3] 최병준, 서태원, "키보드를 모방하는 BadUSB를 차단하기 위한 보안 프로그램," 정보보호학회논문지, 제26권, 제6호, pp. 1483-1492, 2016.

[4] FIRST, "Common Vulnerability Scoring System v3.1: Specification Document," Forum of Incident Response and Security Teams, Jun. 2019.

[5] LOLBAS Project, "Living Off The Land Binaries, Scripts and Libraries," https://lolbas-project.github.io/, (accessed Sep. 22, 2026).

1 Hak5, "USB Rubber Ducky Payloads," GitHub, https://github.com/hak5/usbrubberducky-payloads (accessed Sep. 2026)

2 Red Canary, "Atomic Red Team," GitHub, https://github.com/redcanaryco/atomic-red-team (accessed Sep. 2026)